Data Link Layer

Contents

- Framing
- Flow and Error Control
- Error Detection and Correction
- Types of Errors - Redundancy, LRC, CRC, Checksum

Introduction

- The Data Link Control (DLC) deals with procedures for communication between two adjacent nodes
- Node-to-Node communication no matter whether the link is dedicated or broadcast.
- Data link control functions include framing and flow & error control.

Framing

FRAMING

- The data link layer needs to pack bits into frames, so that each frame is distinguishable from another.
- Our postal system practices a type of framing.
- The simple act of inserting a letter into an envelope separates one piece of information from another; the envelope serves as the delimiter.

Contd...

- Framing in the data-link layer separates a message from one source to a destination by adding a sender address and a destination address.
- The destination address defines where the packet is to go; the sender address helps the recipient acknowledge the receipt.
- Although the whole message could be packed in one frame, that is not normally done.

Contd...

- One reason is that a frame can be very large, making flow and error control very inefficient.
- When a message is carried in one very large frame, even a single-bit error would require the retransmission of the whole frame.
- When a message is divided into smaller frames, a single-bit error affects only that small frame.

Frame Size

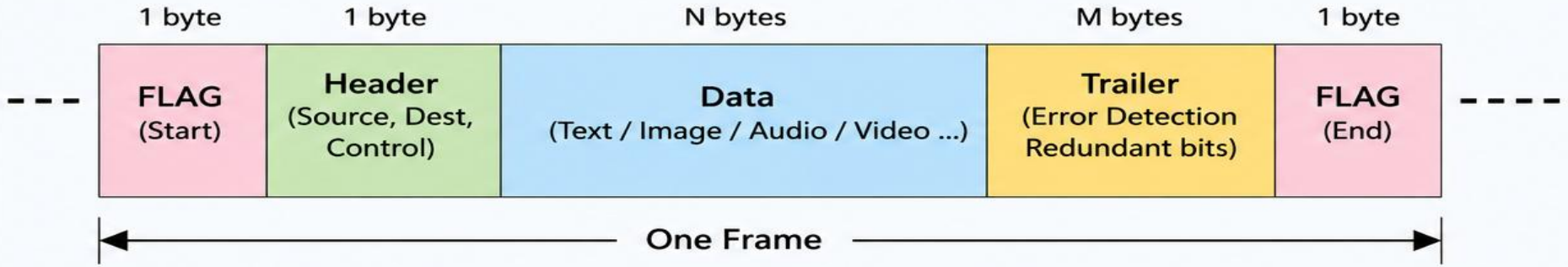
- Frames can be of fixed or variable size.
- In fixed-size framing, there is no need for defining the boundaries of the frames; the size itself can be used as a delimiter.
- Two approaches
  - Character-oriented approach
  - Bit-oriented approach

Character-Oriented (Byte-Oriented) Framing

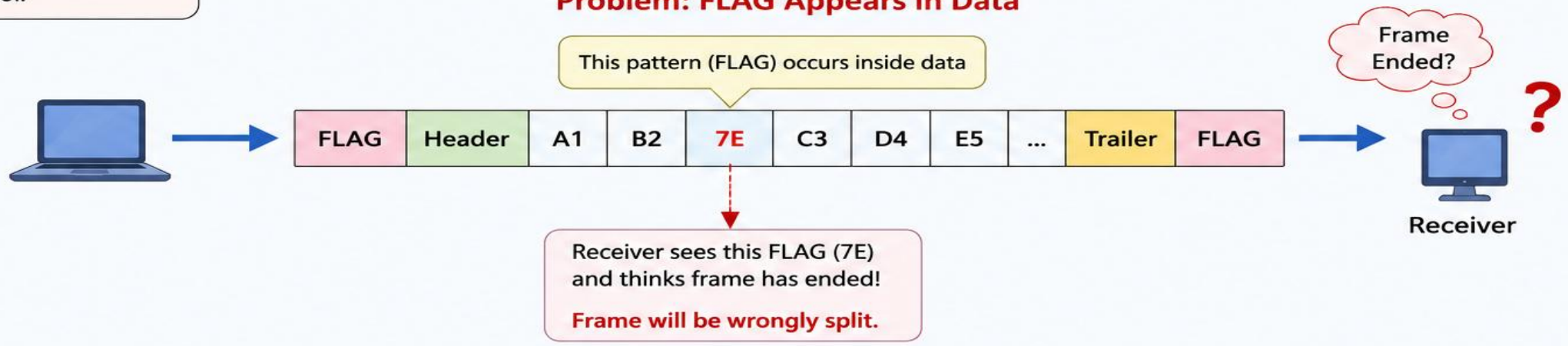
Data, header and trailer are in multiples of 8 bits (1 byte).

Key Points

- Data, header, trailer are in 8-bit characters (multiples of 8 bits).
- An 8-bit **FLAG** is added at the beginning and end to mark frame boundaries.
- **FLAG** is a special character (pattern) chosen by the protocol.



Problem: FLAG Appears in Data



Receiver sees this FLAG (7E) and thinks frame has ended!

Frame will be wrongly split.

Solution

Use methods like:

- Byte Stuffing
- Character Escaping
- or Length Counting

→ These ensure that **FLAG** in data is not mistaken for frame boundary.

Example

FLAG pattern = 7E (hex) (01111110 in binary)

Not used as normal text.

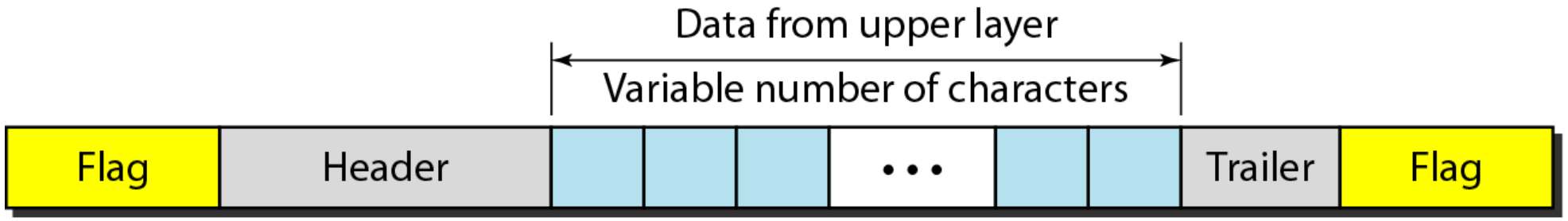
Character-Oriented Framing

- In character-oriented (or byte-oriented) framing, data to be carried are 8-bit characters from a coding system such as ASCII.
- The header - carries the source and destination addresses and other control information, and the trailer, which carries error detection redundant bits, are also multiples of 8 bits.
- To separate one frame from the next, an 8-bit (1-byte) flag is added at the beginning and the end of a frame.

CONTD…

- The flag, composed of protocol-dependent special characters, signals the start or end of a frame.
- The flag could be selected to be any character not used for text communication.
- Other types of information such as graphs, audio, and video; any character used for the flag could also be part of the information.
- The receiver, when it encounters this pattern in the middle of the data, thinks it has reached the end of the frame.

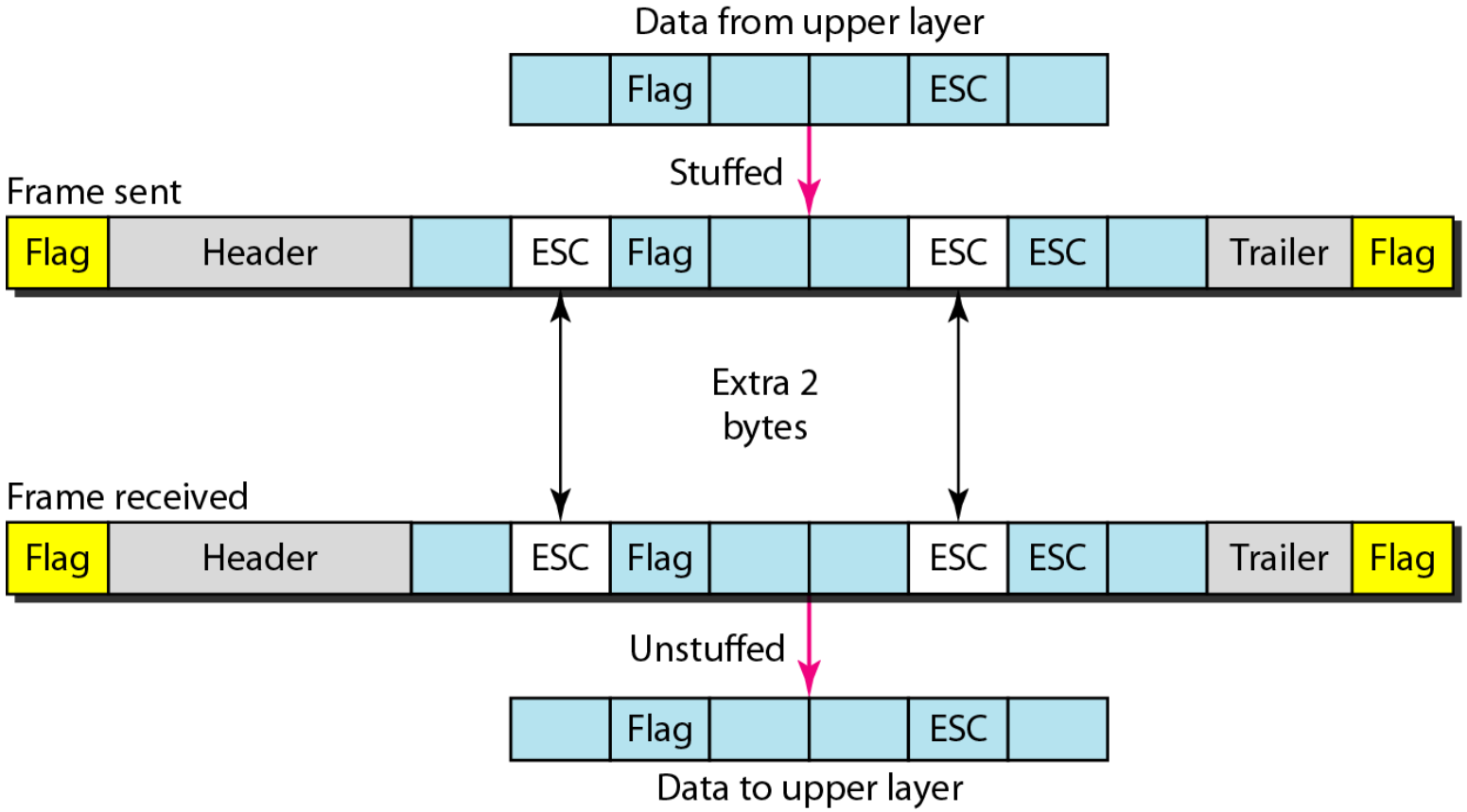
A frame in a character-oriented protocol



Byte stuffing and unstuffing

- A byte-stuffing strategy was added to character-oriented framing.
- In byte stuffing (or character stuffing), a special byte is added to the data section of the frame when there is a character with the same pattern as the flag.
- The data section is stuffed with an extra byte. This byte is usually called the escape character (ESC) and has a predefined bit pattern.
- Whenever the receiver encounters the ESC character, it removes it from the data section and treats the next character as data, not as a delimiting flag

Byte stuffing and unstuffing



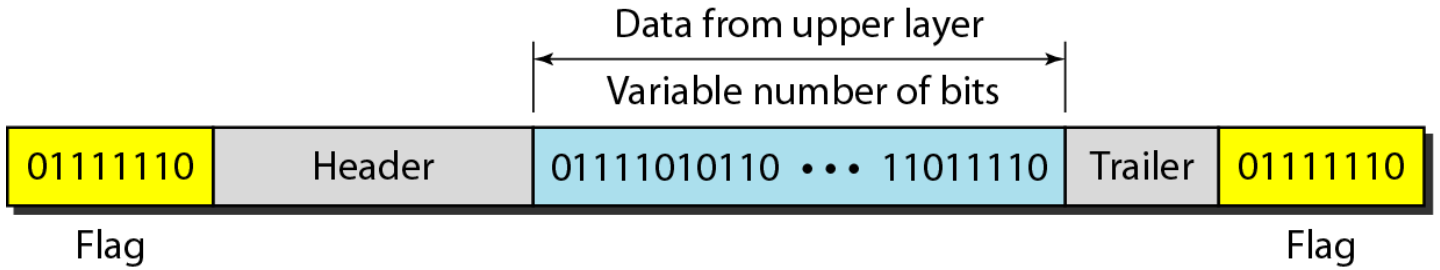
Note

Byte stuffing is the process of adding 1 extra byte whenever there is a flag or escape character in the text.

Bit-Oriented Framing

- In bit-oriented framing, the data section of a frame is a sequence of bits to be interpreted by the upper layer as text, graphic, audio, video, and so on
- Most protocols use a special 8-bit pattern flag, 01111110, as the delimiter to define the beginning and the end of the frame.

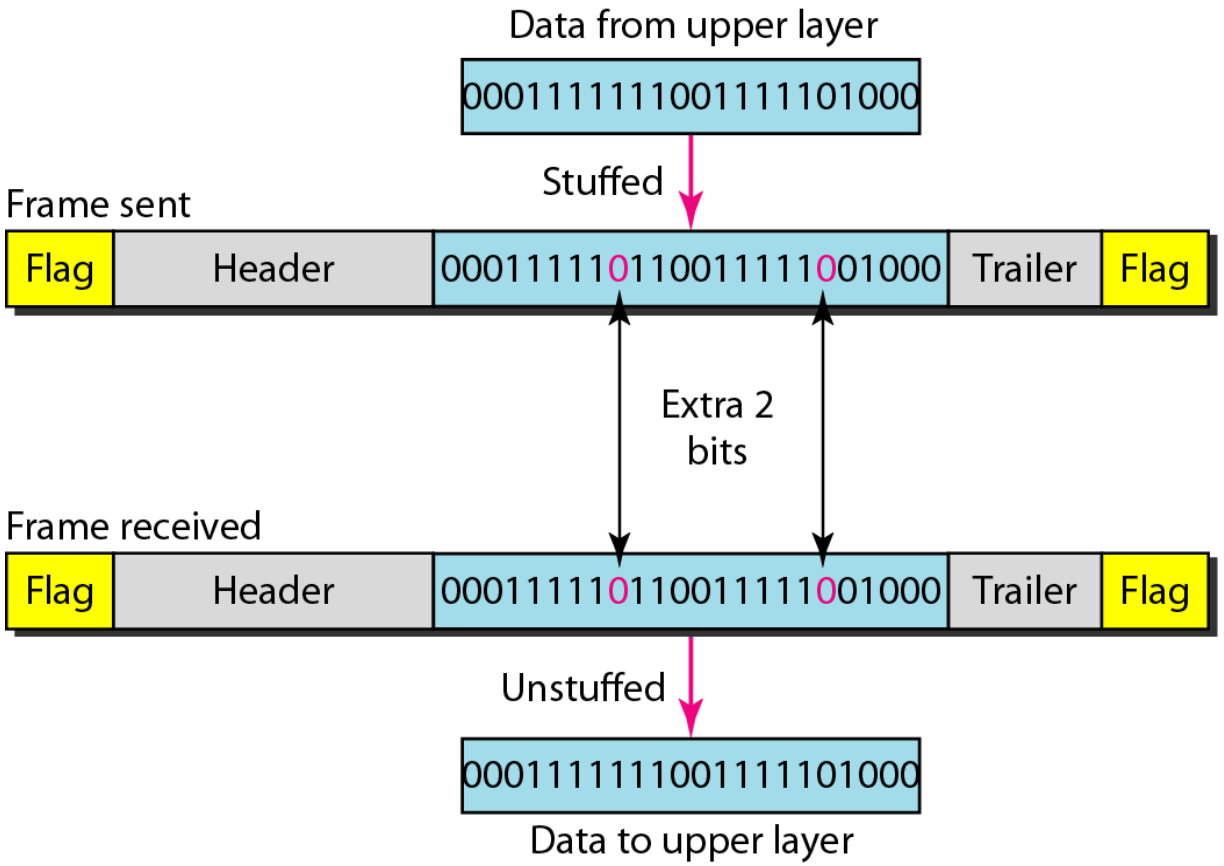
A frame in a bit-oriented protocol



Note

Bit stuffing is the process of adding one extra 0 whenever five consecutive 1s follow a 0 in the data, so that the receiver does not mistake the pattern 0111110 for a flag.

Bit stuffing and unstuffing



Flow and Error Control

FLOW AND ERROR CONTROL

The most important responsibilities of the data link layer are flow control and error control. Collectively, these functions are known as data link control.

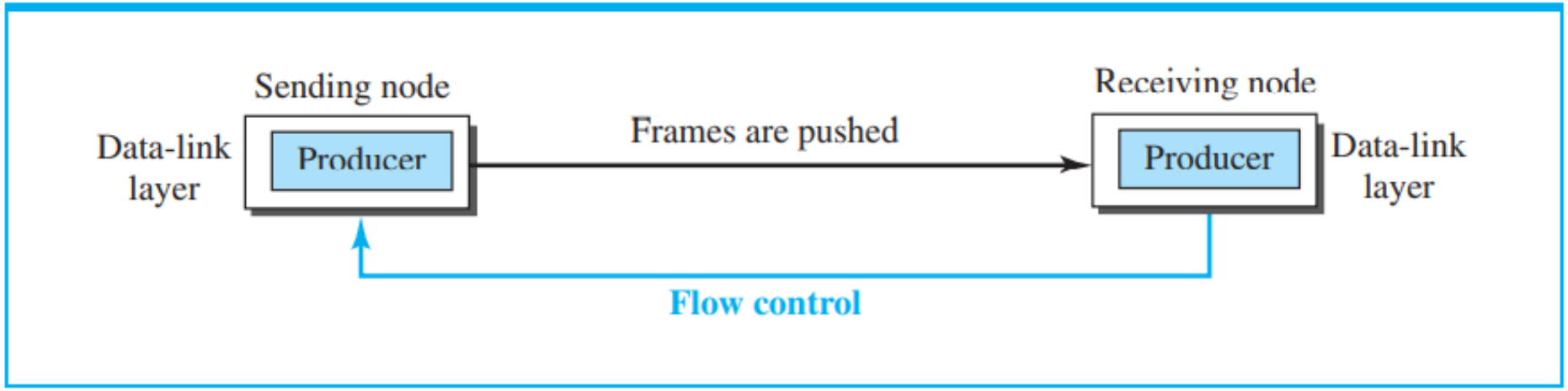
Note

Flow control refers to a set of procedures used to restrict the amount of data that the sender can send before waiting for acknowledgment.

Note

Error control in the data link layer is based on automatic repeat request, which is the retransmission of data.

Flow control at the data-link layer



FLOW CONTROL & BUFFERS AT THE DATA LINK LAYER

FLOW CONTROL

Ensures the sender does not overwhelm the receiver by sending too much data too fast.

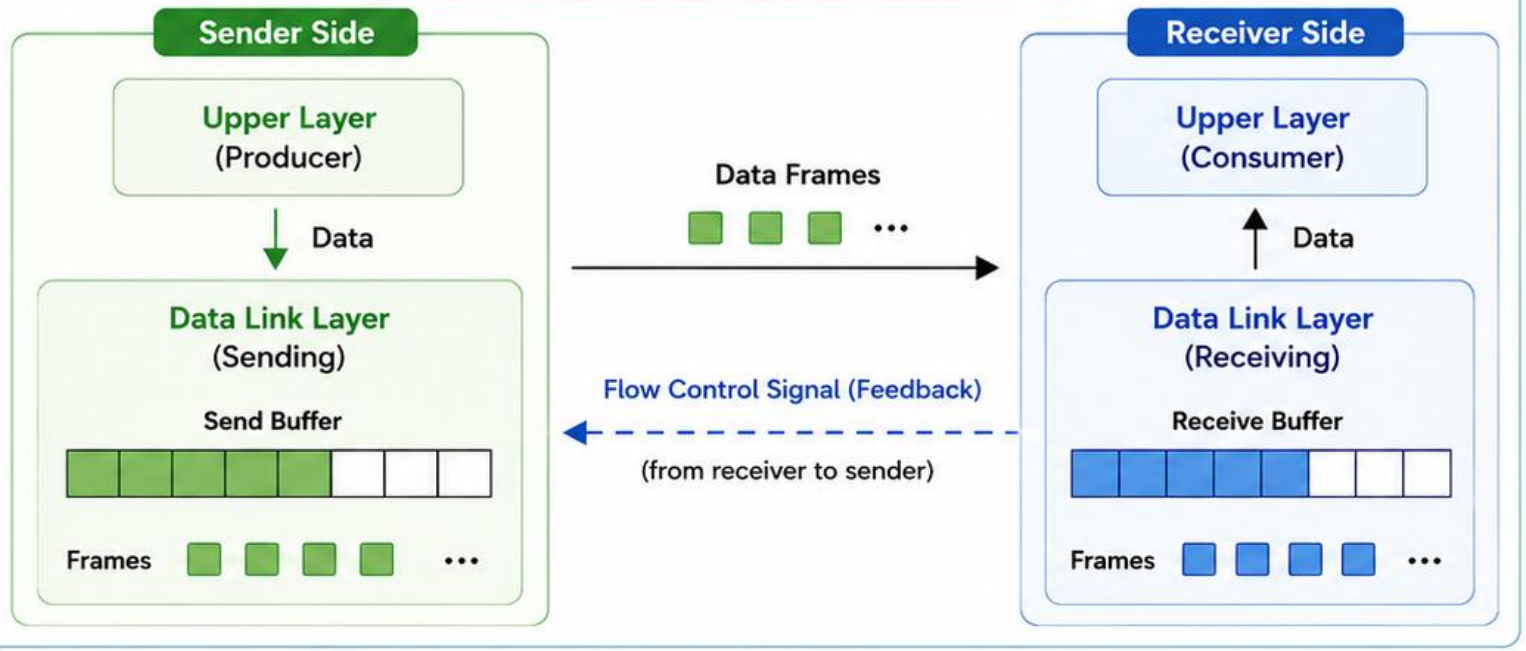


BUFFERS

Buffers are temporary storage areas at the sender and receiver to hold frames.



FLOW CONTROL USING BUFFERS



Frame from Sender to Receiver

Frame stored in Receiver Buffer

READY (Buffer has space)

STOP (Buffer is full)

HOW FLOW CONTROL WORKS

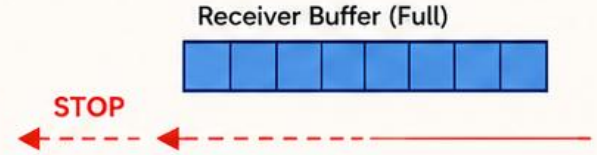
1 BUFFER HAS SPACE

- Receiver buffer has space.
- Receiver sends READY signal.



2 BUFFER IS FULL

- Receiver buffer is full.
- Receiver sends STOP signal.
- Sender stops sending more frames.



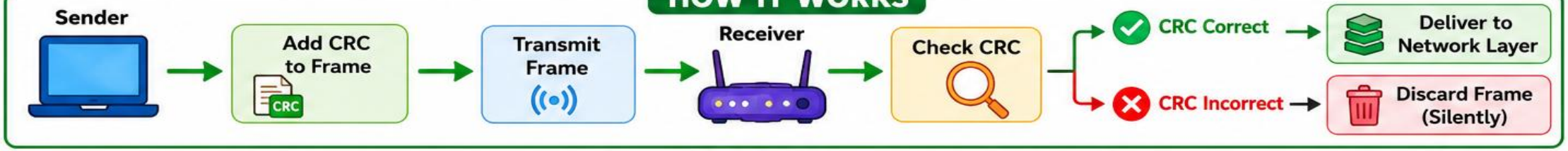
Buffers

- One of the solutions is normally to use two buffers; one at the sending data-link layer and the other at the receiving data-link layer.
- A buffer is a set of memory locations that can hold packets at the sender and receiver.
- The flow control communication can occur by sending signals from the consumer to the producer.
- When the buffer of the receiving data-link layer is full, it informs the sending data-link layer to stop pushing frames.

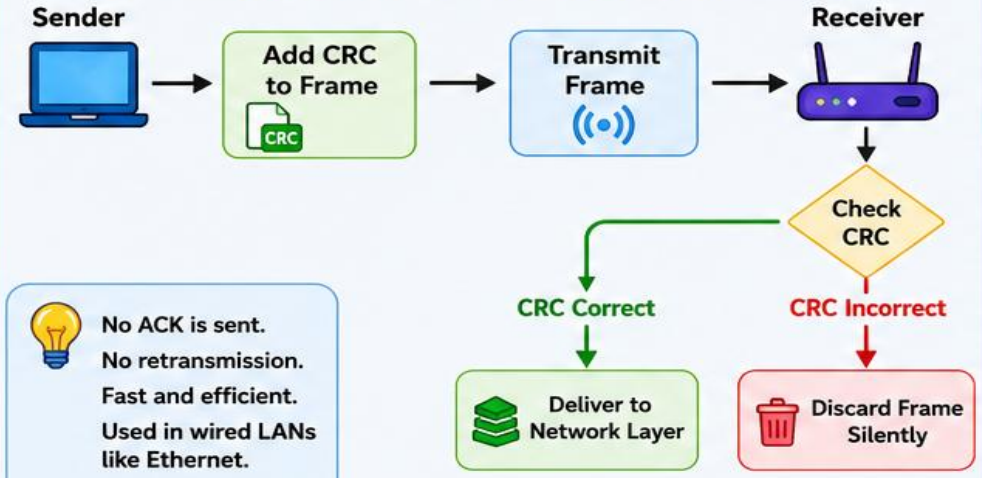
ERROR CONTROL AT THE DATA-LINK LAYER

Physical layer is not fully reliable. Error control at **Data-Link layer** prevents corrupted packets from reaching the **Network layer.**

HOW IT WORKS

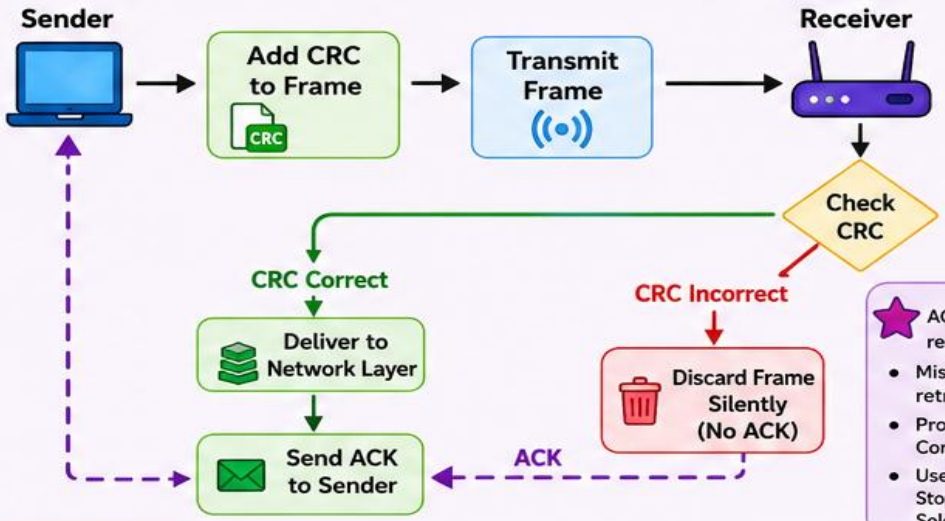


METHOD 1: ERROR DETECTION ONLY
(Used in Ethernet / Wired LANs)



No ACK is sent.
No retransmission.
Fast and efficient.
Used in wired LANs like Ethernet.

METHOD 2: ERROR DETECTION + ACKNOWLEDGMENT
(Used in Reliable Data-Link Protocols)



ACK confirms correct reception.

- Missing ACK → sender retransmits.
- Provides both Flow Control and Error Control.
- Used in protocols like Stop-and-Wait, Go-Back-N, Selective Repeat.

COMPARISON

Feature	Method 1: Error Detection Only	Method 2: Error Detection + Acknowledgment
CRC Error Detection	✓	✓
ACK Sent	✗	✓
Retransmission	✗	✓ (when ACK is missing)
Flow Control	✗	✓
Typical Use	Ethernet (Wired LANs)	Reliable Data-Link Protocols (Stop-and-Wait, Go-Back-N, Selective Repeat)

Error Control

- Physical layer is not fully reliable - error control is implemented at data-link layer to prevent the receiving node from delivering corrupted packets to its network layer.
- Error control at the data-link layer is normally very simple and implemented using one of the following two methods.
- In both methods, a CRC is added to the frame header by the sender and checked by the receiver.

CONTD...

- In the first method, if the **frame is corrupted**, it is silently **discarded**; if it is not corrupted, the packet is delivered to the network layer. This method is used mostly in **wired LANs such as Ethernet.**
- In the second method, if the **frame is corrupted**, it is silently **discarded**; if it is not corrupted, an **acknowledgment is sent** (for the purpose of both flow and error control) to the sender.

Combination of Flow and Error Control

- Flow and error control can be combined.
- In a simple situation, the acknowledgment that is sent for flow control can also be used for error control to tell the sender the packet has arrived uncorrupted.
- The lack of acknowledgment means that there is a problem in the sent frame.
- A frame that carries an acknowledgment is normally called an ACK to distinguish it from the data frame.

Error Detection and Correction

Note

Data can be corrupted during transmission. Some applications require that errors be detected and corrected.

INTRODUCTION

Error detection and correction

- Types of Errors
- Redundancy
- Detection Versus Correction

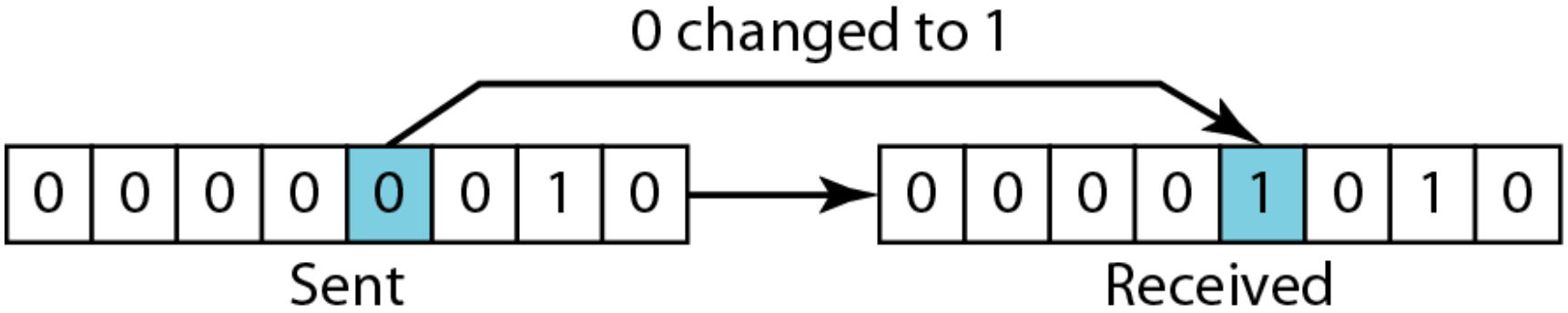
Types of Errors

- Whenever bits flow from one point to another, they are subject to unpredictable changes because of interference.
- This interference can change the shape of the signal.
- The term single-bit error means that only 1 bit of a given data unit (such as a byte, character, or packet) is changed from 1 to 0 or from 0 to 1.
- The term burst error means that 2 or more bits in the data unit have changed from 1 to 0 or from 0 to 1.

Note

In a single-bit error, only 1 bit in the data unit has changed.

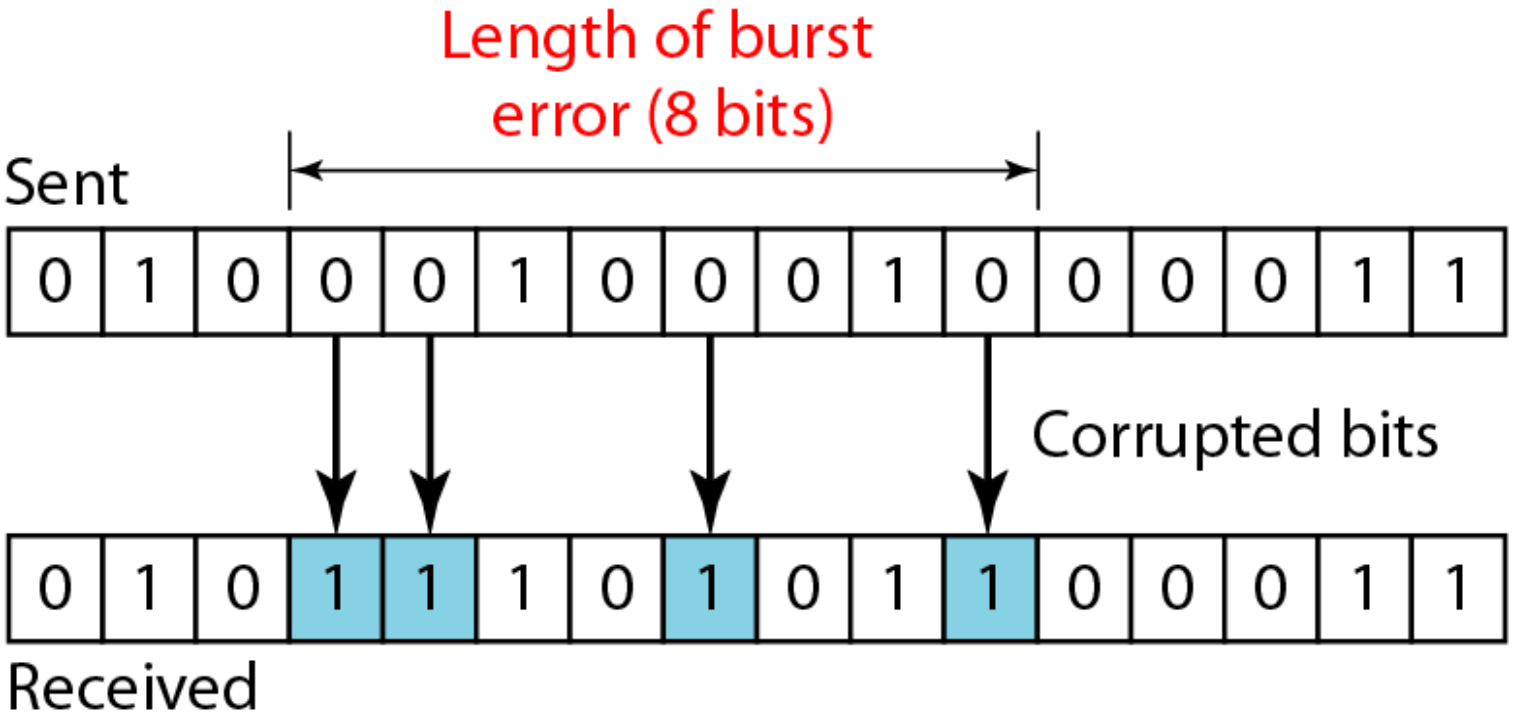
Single-bit error



Note

A burst error means that 2 or more bits in the data unit have changed.

Burst error of length 8



Note

To detect or correct errors, we need to send extra (redundant) bits with data.

The structure of encoder and decoder

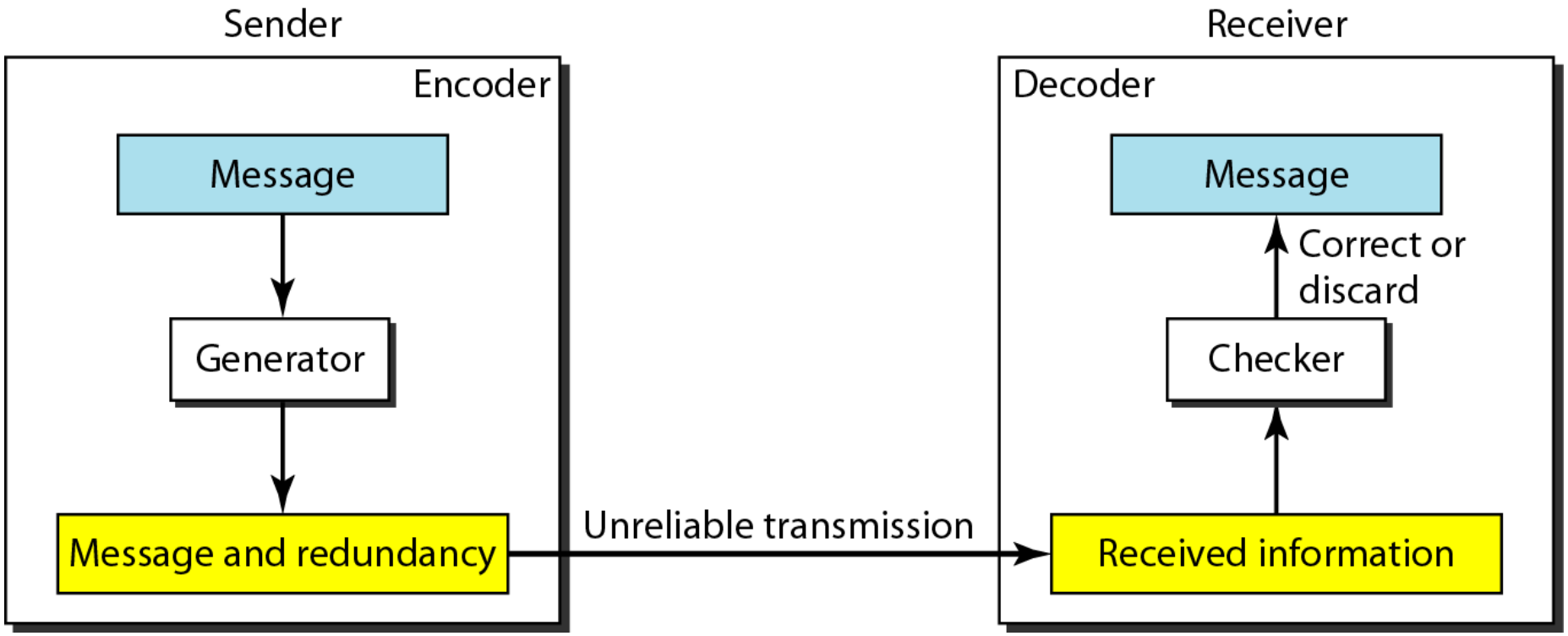


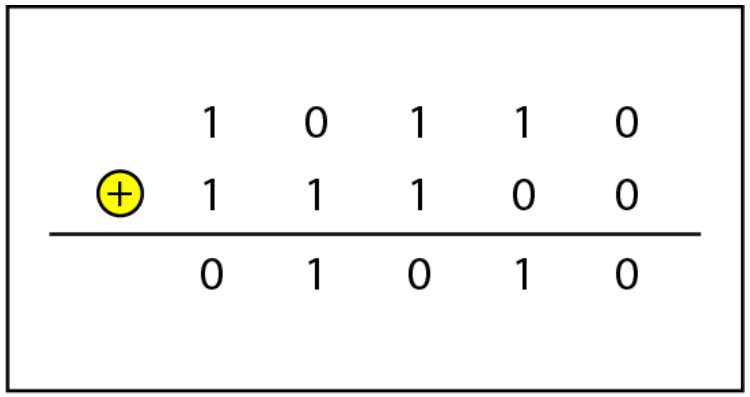
Figure 10.4 *XORing of two single bits or two words*



a. Two bits are the same, the result is 0.



b. Two bits are different, the result is 1.

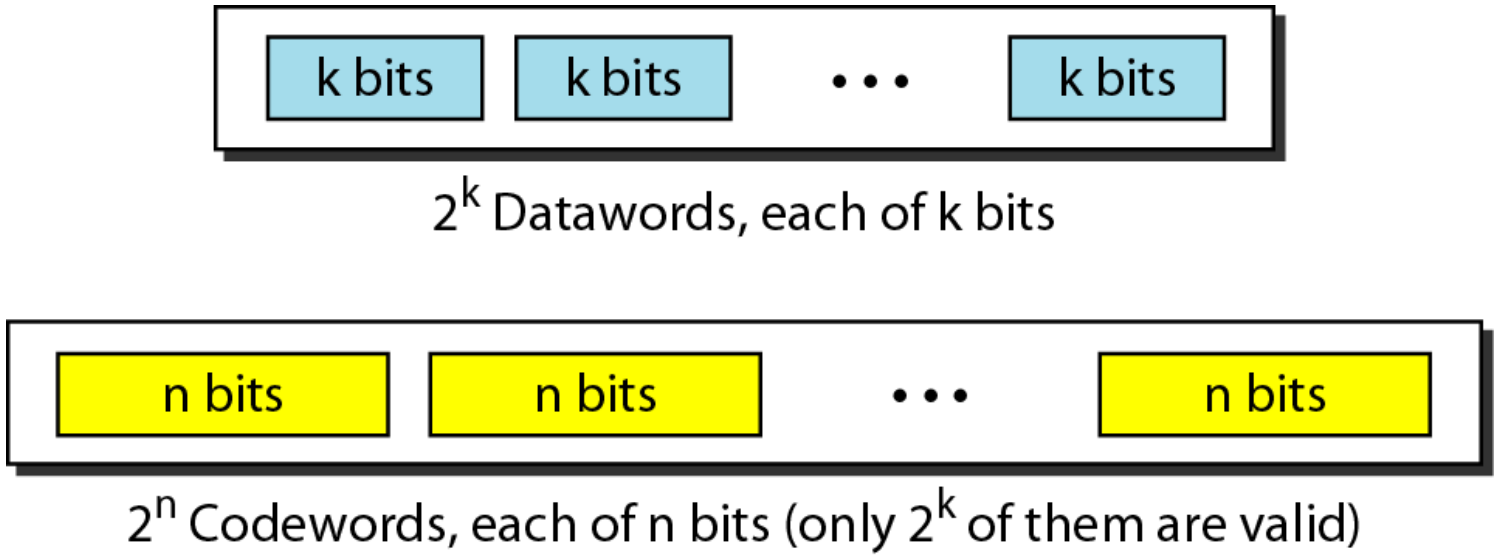


c. Result of XORing two patterns

BLOCK CODING

- In block coding, divide the message into blocks, each of k bits, called datawords.
- Add r redundant bits to each block to make the length n = k + r.
- The resulting n-bit blocks are called codewords.

Datawords and codewords in block coding



Example 10.1

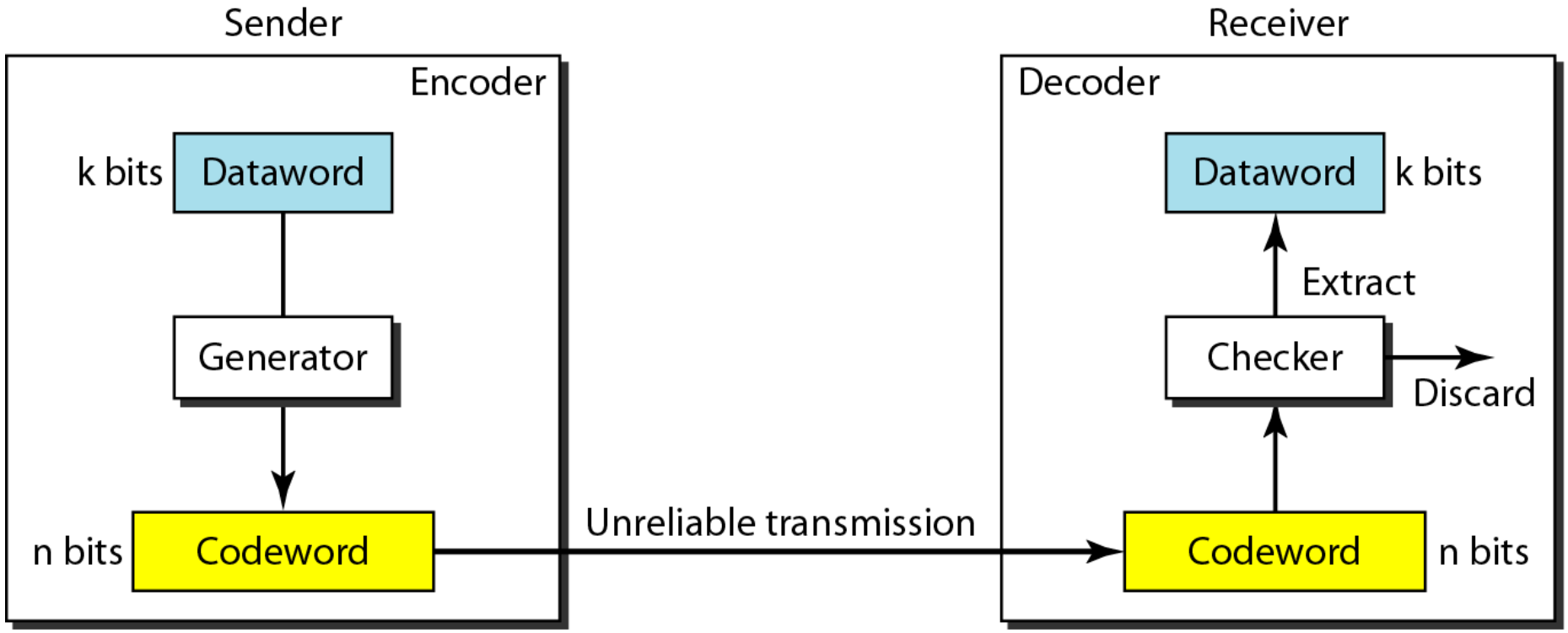
The 4B/5B block coding is a good example of this type of coding. In this coding scheme, $k = 4$ and $n = 5$.

- $2^k = 16$ datawords and $2^n = 32$ codewords.
- 16 out of 32 codewords are used for message transfer and the rest are either used for other purposes or unused.

Error Detection

- Enough redundancy is added to detect an error.
- The receiver knows an error occurred but does not know which bit(s) is(are) in error.
- Has less overhead than error correction.

Process of error detection in block coding



Example 10.2

Let us assume that k = 2 and n = 3. Table 10.1 shows the list of datawords and codewords. Later, we will see how to derive a codeword from a dataword.

Assume the sender encodes the dataword 01 as 011 and sends it to the receiver. Consider the following cases:

1. The receiver receives 011. It is a valid codeword. The receiver extracts the dataword 01 from it.

2. The codeword is corrupted during transmission, and 111 is received. This is not a valid codeword and is discarded.
3. The codeword is corrupted during transmission, and 000 is received. This is a valid codeword. The receiver incorrectly extracts the dataword 00. Two corrupted bits have made the error undetectable.

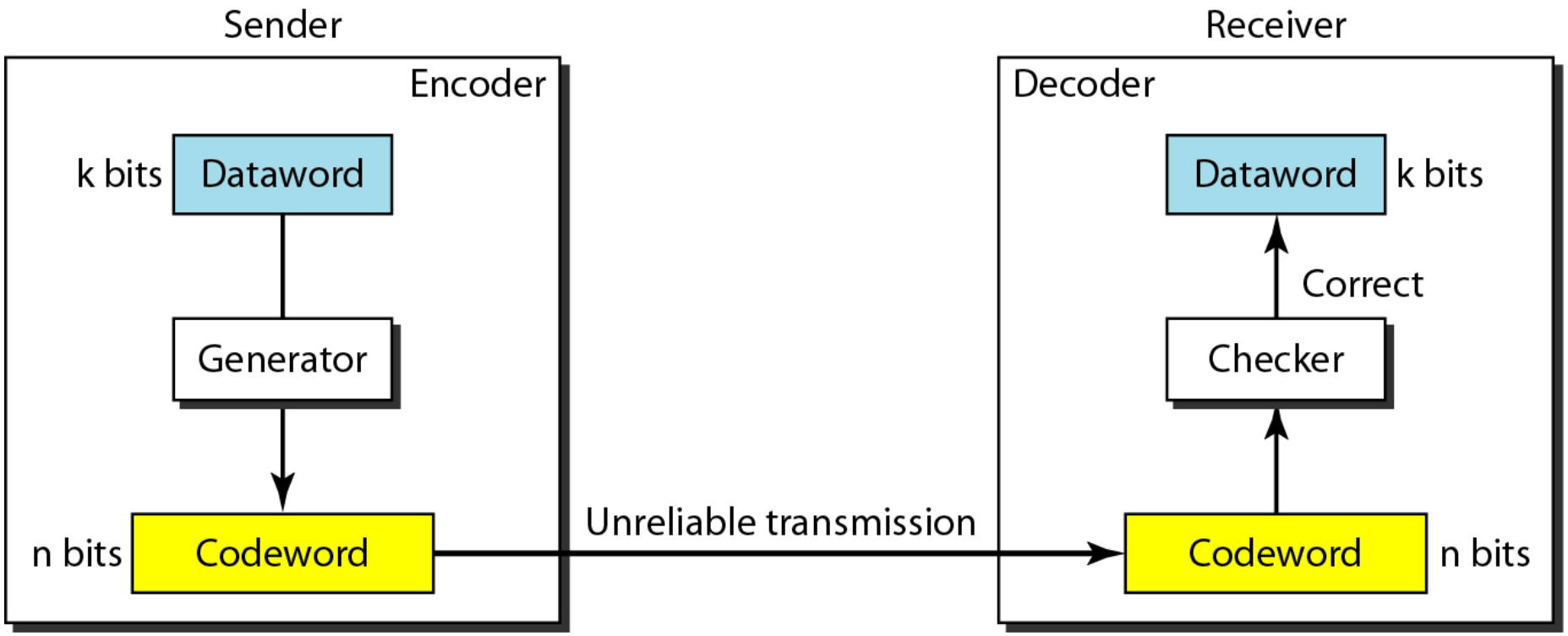
A code for error detection (Example)

Datawords	*Codewords*
00	000
01	011
10	101
11	110

Note

An error-detecting code can detect only the types of errors for which it is designed; other types of errors may remain undetected.

Structure of encoder and decoder in error correction



Let us add more redundant bits to previous Example to see if the receiver can correct an error without knowing what was actually sent. We add 3 redundant bits to the 2-bit dataword to make 5-bit codewords. Table shows the datawords and codewords. Assume the dataword is 01. The sender creates the codeword 01011. The codeword is corrupted during transmission, and 01001 is received. First, the receiver finds that the received codeword is not in the table. This means an error has occurred. The receiver, assuming that there is only 1 bit corrupted, uses the following strategy to guess the correct dataword.

Example (continued)

1. Comparing the received codeword with the first codeword in the table (01001 versus 00000), the receiver decides that the first codeword is not the one that was sent because there are two different bits.
2. By the same reasoning, the original codeword cannot be the third or fourth one in the table.
3. The original codeword must be the second one in the table because this is the only one that differs from the received codeword by 1 bit. The receiver replaces 01001 with 01011 and consults the table to find the dataword 01.

A code for error correction

Dataword	*Codeword*
00	00000
01	01011
10	10101
11	11110